

Docker Compose

Recall:

- **Docker image** = is a read-only blueprint or template used to create containers.
 - Think of it like a recipe for making a cake
- **Docker Container** = is a running instance of an image

`docker network` = Allows containers to communicate with each other and with the external world.

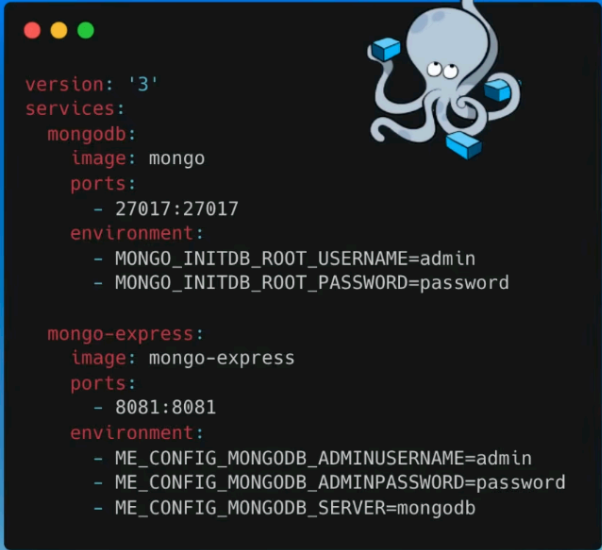
`docker network create {name}` = to create a new user-defined network.

`docker network ls`

```
docker run -d \  
-p 27017:27017 \  
-e MONGO_INITDB_ROOT_USERNAME=admin \  
-e MONGO_INITDB_ROOT_PASSWORD=supersecret \  
--network mongo-network \  
--name mongodb \  
mongo
```

Option	Meaning
<code>docker run</code>	Start a container
<code>-d</code>	Run in background (detached mode)
<code>-p 27017:27017</code>	Expose MongoDB port
<code>-e</code>	Set environment variables
<code>--network mongo-network</code>	Attach to Docker network
<code>--name mongodb</code>	Container name
<code>mongo</code>	MongoDB image

Here's where the docker compose becomes handy



The image shows a Docker Compose configuration file for MongoDB and Mongo-Express. The configuration is written in a dark-themed terminal window. A cartoon octopus is visible in the top right corner of the terminal window. The configuration defines two services: 'mongodb' and 'mongo-express'. The 'mongodb' service uses the 'mongo' image, maps port 27017 to 27017, and sets environment variables for the root user and password. The 'mongo-express' service uses the 'mongo-express' image, maps port 8081 to 8081, and sets environment variables for the admin user, password, and the MongoDB server address.

```
version: '3'
services:
  mongodb:
    image: mongo
    ports:
      - 27017:27017
    environment:
      - MONGO_INITDB_ROOT_USERNAME=admin
      - MONGO_INITDB_ROOT_PASSWORD=password

  mongo-express:
    image: mongo-express
    ports:
      - 8081:8081
    environment:
      - ME_CONFIG_MONGODB_ADMINUSERNAME=admin
      - ME_CONFIG_MONGODB_ADMINPASSWORD=password
      - ME_CONFIG_MONGODB_SERVER=mongodb
```

- ✓ Helps to structure your commands

Simplifies container management

- ✓ Easier to make changes, and see current configuration
- ✓ Declarative approach: defining the desired state

"X" as code " concept:

- Code that defines how your services should run
- Code can be versioned
- Easier collaboration

One more example of docker compose

```

version: '3.1'
services:

  mongodb:
    image: mongo
    ports:
      - 27017:27017
    environment:
      MONGO_INITDB_ROOT_USERNAME: admin
      MONGO_INITDB_ROOT_PASSWORD: supersecret

  mongo-express:
    image: mongo-express
    ports:
      - 8081:8081
    environment:
      ME_CONFIG_MONGODB_ADMINUSERNAME: admin
      ME_CONFIG_MONGODB_ADMINPASSWORD: supersecret
      ME_CONFIG_MONGODB_SERVER: mongodb

```

`docker-compose -f {name}.yaml up`

`docker-compose -f {name}.yaml down` = Stops containers and removes containers, networks, volumes and image created by 'up'

`docker ps` = list running containers on the system

`docker ps -a` = show all containers

`docker container ls` = list your running containers.

`docker container ls -a` = list all the running containers including those that are stopped

Instead of using the `docker-compose -f {name}.yaml down`, we can temporarily stop the compose by `docker-compose -f {name}.yaml stop` and resume it by calling `docker-compose -f {name}.yaml start`

- Stops the running containers
 - Keeps:
 - containers

- networks
- volumes
- container state

Rename the project

```
docker-compose --project-name {name} -f {name}.yaml up
```

Main use case of Docker Compose

- To define and manage services (containers) that make up your app
- So they can be run together in an isolated environment
- Easy to run and clean up your entire app

Limitations

- Well-suited for local development and small-scale deployment
- Designed to run containers on a single host system
- Engineers still need to operate the containers manually.

Kubernetes to the rescue\

To build an image

```
docker build -t {name}
```

docker compose pull` = pull the images

Push the compose to the docker hub

```
docker compose build
```

Authenticate

```
docker login
```

Push it to docker hub

```
docker compose push
```